

Valentini Scalper: Complete Build & Replication Guide

Executive Summary

The **Valentini Scalper** is a professional-grade order flow trading system implementing Fabio Valentini's Triple-A methodology (Absorption → Accumulation → Aggression) for cryptocurrency scalping. This document provides complete technical specifications, architecture details, and step-by-step replication instructions for deploying a production-ready automated trading platform.

Technology Stack: Node.js/TypeScript backend, React frontend, SQLite database, Binance market data integration, FastAPI Python engine (optional)

Key Features: Real-time range bar charting, volume profile analysis, VWAP bands, absorption detection, Triple-A state machine, paper trading OMS, live SSE streaming, configurable risk management

Table of Contents

1. System Architecture Overview
 2. Core Components Deep Dive
 3. Data Flow & Processing Pipeline
 4. Trading Strategy Implementation
 5. Frontend Interface Design
 6. Deployment & Configuration
 7. Testing & Validation
 8. Extending the System
-

1. System Architecture Overview

1.1 High-Level Architecture

The system follows a three-tier architecture with clear separation of concerns:

Layer	Technology	Responsibility
Frontend (Client)	React + TypeScript + Vite	User interface, charting, real-time updates
Backend (Server)	Node.js + Express + TypeScript	API, market data, strategy engine, database
Data Store	SQLite + Drizzle ORM	Trades, configuration, persistence
Market Data	Binance API + WebSocket	Real-time klines, order book, ticker

Table 1: System architecture layers

1.2 Technology Rationale

Why Node.js/TypeScript?

- Single language across frontend and backend (code reuse)
- Excellent async/event-driven model for real-time data streams
- Fast prototyping with strong typing (TypeScript)
- Rich ecosystem for financial data processing

Why SQLite?

- Zero-configuration embedded database
- ACID compliance for trade integrity
- Write-Ahead Logging (WAL) mode for concurrent access
- Perfect for single-server deployments

Why React?

- Component-based architecture matches trading UI patterns
- Hooks enable clean state management for real-time updates
- TanStack Query simplifies API integration
- Virtual DOM efficient for high-frequency chart updates

1.3 Project Structure

valentini-scalper/

- |— package.json # Dependencies and scripts
- |— tsconfig.json # TypeScript config (client)
- |— tsconfig.server.json # TypeScript config (server)
- |— vite.config.ts # Vite bundler config
- |— tailwind.config.js # Tailwind CSS config
- |— drizzle.config.ts # Drizzle ORM config
- |— dev.db # SQLite database (generated)
- |
- |— shared/ # Shared types (client + server)
 - |— schema.ts # Database schema definitions
- |
- |— server/ # Backend (Node.js/Express)
 - |— index.ts # Server entry point
 - |— routes.ts # API routes + SSE
 - |— db.ts # Database initialization
 - |— storage.ts # Database query helpers
 - |— binance-client.ts # Binance API wrapper
 - |— range-bar-generator.ts # Range bar construction
 - |— strategy-engine.ts # Core strategy logic
- |
- |— client/ # Frontend (React)
 - |— index.html # HTML entry
 - |— src/
 - |— main.tsx # React entry
 - |— App.tsx # Root component
 - |— index.css # Global styles
 - |— lib/
 - |— queryClient.ts # API client
 - |— pages/
 - |— dashboard.tsx # Main dashboard
 - |— components/
 - |— TradingChart.tsx
 - |— VolumeProfilePanel.tsx
 - |— OrderBookPanel.tsx
 - |— SignalPanel.tsx
 - |— TradeHistory.tsx
 - |— StrategySettings.tsx

```
|
|— python_backend/ # Optional Python engine
|— main.py # FastAPI app
|— requirements.txt # Python dependencies
|— engine/
|— config.py
|— detectors/
|   |— volume_profile.py
|   |— triple_a.py
|   |— absorption.py
|— oms/
|— paper_oms.py
```

2. Core Components Deep Dive

2.1 Range Bar Generator

Purpose: Convert time-based candlesticks (1-minute klines) into range bars with fixed price movement.

Algorithm:

Range bars close when price movement reaches a threshold (e.g., 50 points), regardless of time. This filters out time-based noise and focuses on price action.

Implementation Logic:

1. Calculate auto range size using ATR(14) if not specified
2. Simulate tick-by-tick price movement within each 1m candle
3. Track current bar's open, high, low, close, volume
4. When range threshold reached ($\text{high} - \text{low} \geq \text{rangeSize}$), close bar
5. Distribute volume proportionally across generated bars

Key Function: `buildRangeBars(klines, rangeSize)`

Input: Array of 1-minute Binance klines

Output: Array of range bars with `isComplete` flag

Auto Range Calculation:

$$\text{ATR}(14) = \frac{1}{14} \sum_{i=1}^{14} \max(H_i - L_i, |H_i - C_{i-1}|, |L_i - C_{i-1}|)$$

RangeSize = min(tick ∈ [5, 10, 25, 50, 100, 200, 500] | tick ≥ 0.3

Code Location: server/range-bar-generator.ts

Edge Cases Handled:

- Incomplete bars (marked with isComplete: false)
- Zero volume periods (proportional distribution)
- Gap movements (tick simulation ensures continuity)

2.2 Volume Profile Construction

Purpose: Build horizontal volume distribution to identify high-activity price levels (POC, VAH, VAL).

Algorithm:

1. Determine price range: $[\min(P), \max(P)]$
2. Divide into buckets with width = range size
3. For each bar, allocate buy/sell volume to corresponding bucket
4. Calculate Point of Control (POC) = bucket with max total volume
5. Calculate Value Area (68% volume around POC) using expansion algorithm
6. Derive VAH (Value Area High) and VAL (Value Area Low)

Value Area Expansion:

Starting from POC, iteratively expand to adjacent buckets (higher or lower volume first) until 68% of total volume captured.

Data Structure:

```
interface VPLevel {
  price: number;
  buyVol: number;
  sellVol: number;
  total: number; // buyVol + sellVol
  delta: number; // buyVol - sellVol
}
```

```
interface VolumeProfile {
levels: VPLevel[];
poc: number; // Point of Control
vah: number; // Value Area High
val: number; // Value Area Low
step: number; // Bucket size
}
```

Code Location: server/strategy-engine.ts → buildVP()

Use Case: POC acts as magnet for price, VAH/VAL define trading ranges for mean-reversion setups.

2.3 VWAP Calculation

Purpose: Volume-weighted average price provides institutional trading levels and directional bias.

Formula:

$$\text{VWAP} = \frac{\sum_{i=1}^n \text{TP}_i \times V_i}{\sum_{i=1}^n V_i}$$

where $\text{TP}_i = \frac{H_i + L_i + C_i}{3}$ (typical price)

Standard Deviation Bands:

$$\sigma = \sqrt{\frac{\sum_{i=1}^n V_i \times (\text{TP}_i - \text{VWAP})^2}{\sum_{i=1}^n V_i}}$$

Upper Band 1 = $\text{VWAP} + \sigma$, Lower Band 1 = $\text{VWAP} - \sigma$

Upper Band 2 = $\text{VWAP} + 2\sigma$, Lower Band 2 = $\text{VWAP} - 2\sigma$

Code Location: server/strategy-engine.ts → calcVwap()

Trading Logic: Price above VWAP = bullish bias, below = bearish bias. Bands define overbought/oversold zones.

2.4 Absorption Detection

Purpose: Identify institutional absorption (large volume with minimal price movement) signaling potential reversals.

Detection Criteria:

- Volume $\geq$ 20-bar average volume $\times$ multiplier (default 1.5)
- Price range (high - low) $\leq$ range size $\times$ threshold (default 0.5)
- Side determined by buy/sell volume ratio
- Strength calculated as normalized volume excess

Algorithm:

1. Calculate 20-bar rolling average volume
2. For each bar, check if volume exceeds threshold AND range is compressed
3. Classify as BUY absorption if buyVolume $>$ 50% total
4. Calculate strength: $\min \left(\frac{\text{volume/avgVol}}{\text{multiplier}} \times 0.5 + 0.5, 1.0 \right)$

Output Structure:

```
interface Absorption {  
  barIndex: number;  
  price: number;  
  volume: number;  
  side: "BUY" | "SELL";  
  strength: number; // 0-1 scale  
}
```

Code Location: server/strategy-engine.ts $\rightarrow$ detectAbsorptions()

Interpretation: BUY absorption at support suggests institutional accumulation (bullish). SELL absorption at resistance suggests distribution (bearish).

2.5 Triple-A State Machine

Purpose: Implement Fabio Valentini's three-phase setup: Absorption $\rightarrow$ Accumulation $\rightarrow$ Aggression.

State Transitions:

Current State	Trigger	Next State
Waiting	Absorption detected	Absorbing
Absorbing	2+ bars elapsed + price near POC	Accumulating
Accumulating	BUY absorption + price above VWAP	Signal (LONG)
Accumulating	SELL absorption + price below VWAP	Signal (SHORT)
Signal	Next bar	Waiting

Table 2: Triple-A state machine transitions

Implementation:

```
private tripleAPhase: "waiting" | "absorbing" | "accumulating" |
"signal";

updateTripleA(bars, absorptions, vp, vwap) {
  const last = bars.at(-1);
  const recentAbs = absorptions.filter(a => a.barIndex >= bars.length -
5);

  if (this.tripleAPhase === "waiting" && recentAbs.length > 0) {
    this.tripleAPhase = "absorbing";
    this.lastAbsorptionIdx = bars.length - 1;
  }
  // ... state progression logic
}
```

Code Location: server/strategy-engine.ts → updateTripleA()

Trading Edge: Combines institutional footprint (absorption) with market structure (POC) and momentum (VWAP breakout) for high-probability entries.

2.6 Signal Generation

Purpose: Generate actionable LONG/SHORT signals with entry, stop-loss, take-profit levels.

Signal Priority Hierarchy:

1. **Triple-A Signal** (highest) — Absorption + accumulation + VWAP breakout
2. **Value Area Fade** — VAL bounce (long) or VAH rejection (short) with delta confirmation
3. **No Trade** — Default when no valid setup present

Position Sizing Logic:

$$\text{Position Size} = \frac{\text{Risk Per Trade}}{\text{Entry} - \text{Stop Loss}}$$

Risk-Reward Validation:

$$\text{R:R Ratio} = \frac{\text{Take Profit} - \text{Entry}}{\text{Entry} - \text{Stop Loss}} \geq \text{Min R:R}$$

Signal Structure:

```
interface Signal {
  type: "LONG" | "SHORT" | "NO_TRADE";
  reason: string;
  entry: number;
  sl: number; // Stop-loss
  tp: number; // Take-profit
  rr: number; // Risk-reward ratio
  confidence: number; // 0-1 scale
  timestamp: number;
}
```

Example Signal Generation (Triple-A LONG):

```
if (lastAbs.side === "BUY" && price > vwap) {
  const entry = price;
  const sl = vp.val - vp.step;
  const tp = entry + (entry - sl) * tpMultiplier;
  const rr = (tp - entry) / (entry - sl);
  if (rr >= minRR) {
    return {
      type: "LONG",
      reason: "Triple-A BUY absorption above VWAP",
      entry, sl, tp, rr,
    }
  }
}
```

```
confidence: lastAbs.strength,  
timestamp: Date.now()  
};  
}  
}
```

Code Location: server/strategy-engine.ts → generateSignal()

3. Data Flow & Processing Pipeline

3.1 Historical Analysis Flow

User Action: Click "Analyze" button in dashboard

Backend Flow:

1. **API Request:** POST /api/analyze with {symbol, rangeSize}
2. **Data Fetch:** Parallel requests to Binance
 - getKlines(symbol, "1m", 1000) — Last 1000 1-minute candles
 - getTicker(symbol) — Current price & 24h change
 - getOrderBook(symbol) — L2 depth (20 levels)
3. **Range Bar Construction:** buildRangeBars(klines, rangeSize)
4. **Strategy Analysis:** StrategyEngine.analyse(bars, rangeSize, ticker)
 - Build volume profile
 - Calculate VWAP & bands
 - Detect absorptions
 - Update Triple-A state
 - Generate signal
5. **Response:** Return complete analysis object to frontend

Frontend Flow:

1. Receive analysis data via API
2. Update state with setData(analysisResult)
3. Re-render components:
 - TradingChart — draws range bars, VP, VWAP, absorptions
 - VolumeProfilePanel — displays level-by-level breakdown
 - SignalPanel — shows current signal with entry/exit levels

- OrderBookPanel — renders bid/ask depth

3.2 Live Streaming Flow (SSE)

User Action: Click "Start Live" button

Backend Flow:

1. **Initialize:** POST /api/live/start
 - Fetch initial 1000 klines for context
 - Calculate auto range size
 - Build initial range bars
 - Subscribe to Binance WebSocket: symbol@kline_1m
2. **WebSocket Handler:** On each 1m kline message
 - Convert kline to range bar(s)
 - Append to sliding window (last 1000 bars)
 - Run strategy analysis
 - Serialize result to JSON
 - Broadcast to all SSE clients: data: {...}\n\n
3. **Client Connection:** GET /api/live (SSE endpoint)
 - Set headers: Content-Type: text/event-stream
 - Keep connection alive
 - Stream analysis updates every 1 minute

Frontend Flow:

1. Open EventSource connection to /api/live
2. Register onmessage handler: parse JSON → setData(result)
3. Chart updates in real-time without page refresh
4. Connection automatically recovers on error

Performance Optimization:

- Sliding window (last 1000 bars) prevents memory growth
- WebSocket reconnection with exponential backoff
- SSE multiplexing supports multiple concurrent viewers
- Chart uses Canvas API for 60fps rendering

3.3 Database Persistence Flow

Trade Lifecycle:

1. **Signal Generated:** Strategy engine produces LONG/SHORT signal
2. **Create Trade:** INSERT into trades table
 - symbol, side, entryPrice, stopLoss, takeProfit
 - quantity (calculated from risk per trade)
 - status = "open"
 - openedAt = timestamp
3. **Monitor Position:** On each bar update, check exit conditions
 - Price hits stop-loss → close with loss
 - Price hits take-profit → close with profit
 - End of session → close at market
4. **Update Trade:** UPDATE trades SET exitPrice, pnl, status, closedAt
5. **Display History:** Query trades via GET /api/trades

Configuration Persistence:

- Strategy parameters stored in strategy_config table
- Updated via POST /api/config
- Loaded on engine initialization
- Survives server restarts

Schema Design:

```
// trades table
{
id: auto-increment primary key
symbol: "BTCUSDT"
side: "LONG" | "SHORT"
entryPrice: 65000.50
exitPrice: 65100.25 (nullable)
stopLoss: 64950.00
takeProfit: 65150.00
quantity: 0.001
status: "open" | "closed" | "cancelled"
pnl: 99.75 (nullable)
signal: JSON string (optional)
```

```
openedAt: timestamp
closedAt: timestamp (nullable)
}
```

```
// strategy_config table
{
id: 1 (singleton)
symbol: "BTCUSDT"
rangeSize: 0 (auto)
absVolumeMultiplier: 1.5
absRangeThreshold: 0.5
maxDailyLosses: 3
riskPerTrade: 100
tpMultiplier: 2.0
minRR: 1.5
useTripleA: 1
useAbsorption: 1
useVwap: 1
updatedAt: timestamp
}
```

4. Trading Strategy Implementation

4.1 Valentini Triple-A Methodology

Philosophy: Institutional order flow creates predictable patterns before directional moves. Triple-A captures this footprint.

Phase 1: Absorption

What: Large volume with minimal price movement

Where: Support (for longs) or resistance (for shorts)

Why: Institutions absorbing supply/demand before accumulation

Detection:

- Volume $\geq 1.5 \times$ average
- Price range $\leq 0.5 \times$ rangeSize
- Side: BUY if buyVol > 50%, else SELL

Phase 2: Accumulation

What: Price consolidates near Point of Control (POC)

Where: Within 2 range steps of POC

Why: Institutions building position at value

Validation:

- 2+ bars elapsed since absorption
- $\text{Price} \in [\text{POC} - 2 \times \text{step}, \text{POC} + 2 \times \text{step}]$

Phase 3: Aggression (Signal)

What: Price breaks above/below VWAP with absorption confirmation

Where: VWAP breakout zone

Why: Institutions pushing price directionally

Entry Conditions (LONG):

- BUY absorption detected
- $\text{Price} > \text{VWAP}$
- $\text{R}:\text{R} \geq 1.5$

Entry Conditions (SHORT):

- SELL absorption detected
- $\text{Price} < \text{VWAP}$
- $\text{R}:\text{R} \geq 1.5$

Stop-Loss Placement:

- LONG: Below VAL - 1 step
- SHORT: Above VAH + 1 step

Take-Profit Calculation:

- $\text{TP} = \text{Entry} \pm (\text{Entry} - \text{SL}) \times \text{tpMultiplier}$ (default 2.0)

4.2 Complementary Setups

Value Area Fade (Mean-Reversion)

LONG Setup:

- Price near VAL
- Price above VWAP

- Cumulative delta positive
- Target: POC

SHORT Setup:

- Price near VAH
- Price below VWAP
- Cumulative delta negative
- Target: POC

ORB Breakout (Opening Range Breakout)

- First 6 range bars define ORB high/low
- Breakout above ORB high + volume confirmation → LONG
- Breakdown below ORB low + volume confirmation → SHORT

4.3 Risk Management

Daily Loss Limit:

```
const dailyTrades = trades.filter(t =>
t.closedAt >= sessionStart && t.pnl < 0
);
if (dailyTrades.length >= maxDailyLosses) {
return NO_TRADE; // Stop trading for the day
}
```

Position Sizing:

$$\text{Quantity} = \frac{\text{Risk Per Trade (USD)}}{\text{Entry Price} - \text{Stop Loss Price}}$$

Max Drawdown Protection:

Track cumulative PnL. If drawdown exceeds threshold (e.g., -5%), halt trading and alert user.

5. Frontend Interface Design

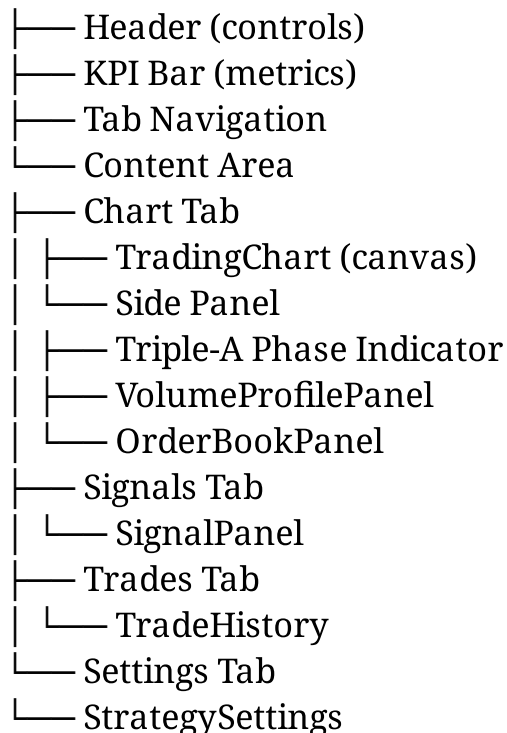
5.1 Dashboard Layout

Structure:

- **Header Bar** — Symbol selector, range size dropdown, Analyze/Live buttons
- **KPI Bar** — Price, Change, Delta, POC, VWAP, VAH, VAL, Phase, Bars, Range
- **Tab Navigation** — Chart, Signals, Trades, Settings
- **Main Content Area** — Tab-specific views
- **Side Panel** (Chart tab only) — Triple-A phase, Volume Profile, Order Book

Component Hierarchy:

Dashboard



5.2 TradingChart Component

Rendering Technology: HTML5 Canvas (manual drawing for performance)

Layers (back to front):

1. Background grid (6 horizontal lines)
2. Volume profile histogram (left side, 60px width)
3. Volume bars (bottom, 60px height)
4. Horizontal levels (POC, VAH, VAL, VWAP, VWAP bands, ORB)
5. Range bars (candlestick-style with border)
6. Absorption bubbles (circles with "A" label)
7. Volume bubbles (top 30% volume bars)
8. Signal arrow (entry point indicator)
9. OHLCV overlay (top-left data box)

Price Scale Calculation:

$$\text{Padding} = (\text{max_price} - \text{min_price}) \times 0.05$$

$$\text{Price to Y} = \text{TOP_PAD} + \frac{\text{HIGH} - \text{price}}{\text{HIGH} - \text{LOW}} \times \text{chartHeight}$$

Volume Scale:

$$\text{Volume Bar Height} = \frac{\text{bar.volume}}{\text{max_volume}} \times \text{volumeHeight}$$

Code Snippet (Range Bar Rendering):

```
view.forEach((b, i) => {
  const x = bx(i);
  const oY = py(b.open), cY = py(b.close);
  const hY = py(b.high), lY = py(b.low);
  const bull = b.close >= b.open;
  const bodyTop = Math.min(oY, cY);
  const bodyH = Math.max(1, Math.abs(cY - oY));

  // Wick
  ctx.strokeStyle = bull ? "#10b981" : "#ef4444";
  ctx.lineWidth = 1;
  ctx.beginPath();
  ctx.moveTo(x + barW / 2, hY);
  ctx.lineTo(x + barW / 2, lY);
  ctx.stroke();
});
```

```
// Body
ctx.fillStyle = bull ? "rgba(16,185,129,0.7)" : "rgba(239,68,68,0.7)";
ctx.fillRect(x, bodyTop, barW, bodyH);
ctx.strokeRect(x, bodyTop, barW, bodyH);
});
```

5.3 Real-Time Updates

EventSource Integration:

```
const toggleLive = async () => {
  if (isLive) {
    await apiRequest("/live/stop", { method: "POST" });
    setIsLive(false);
  } else {
    await apiRequest("/live/start", {
      method: "POST",
      body: JSON.stringify({ symbol, rangeSize })
    });
    setIsLive(true);
    const es = new EventSource("/api/live");
    es.onmessage = (e) => setData(JSON.parse(e.data));
    es.onerror = () => {
      es.close();
      setIsLive(false);
    };
  }
};
```

Auto-Refresh (Polling Fallback):

```
useEffect(() => {
  const id = setInterval(() => {
    if (!isLive) analyze();
  }, 30000); // 30 seconds
  return () => clearInterval(id);
}, [isLive, analyze]);
```

6. Deployment & Configuration

6.1 Installation Steps

Prerequisites:

- Node.js 20+ (LTS recommended)
- npm or pnpm package manager
- Git (for cloning repository)

Clone & Install:

```
git clone https://github.com/your-repo/valentini-scalper.git  
cd valentini-scalper  
npm install
```

Build for Production:

```
npm run build
```

Outputs to dist/ directory

Run Development Server:

```
npm run dev
```

Server starts on
<http://localhost:5000>

Run Production Server:

```
npm start
```

Runs compiled server from dist/

6.2 Environment Configuration

Create .env file (optional):

```
PORT=5000
DB_PATH=./dev.db
BINANCE_API_URL=https://data-api.binance.vision/api/v3
BINANCE_WS_URL=wss://data-stream.binance.vision/ws
```

Default Configuration:

If no .env file, system uses hardcoded defaults in server/binance-client.ts:

```
const BASE = "https://data-api.binance.vision/api/v3";
const WS_BASE = "wss://data-stream.binance.vision/ws";
```

6.3 Database Initialization

Automatic Setup:

On first run, server/db.ts automatically creates SQLite database and tables:

```
const createSQL = CREATE TABLE IF NOT EXISTS trades (...); CREATE
TABLE IF NOT EXISTS strategy_config (...);;
sqlite.exec(createSQL);
```

Manual Migration (if using Drizzle migrations):

```
npx drizzle-kit generate:sqlite
npx drizzle-kit push:sqlite
```

6.4 Binance API Configuration

Using Public Data Endpoints (Default):

System uses data-api.binance.vision — no API keys required.
Supports:

- Market data (klines, ticker, order book)
- WebSocket streams (real-time klines)

Limitations: Cannot execute live trades (read-only).

For Live Trading (Future Enhancement):

1. Register Binance account and create API keys
2. Add to .env:

```
BINANCE_API_KEY=your_api_key  
BINANCE_API_SECRET=your_api_secret
```

3. Update server/binance-client.ts to use authenticated endpoints
4. Implement order execution in OMS module

6.5 Firewall & Network Configuration

Required Outbound Connections:

- HTTPS to data-api.binance.vision (port 443)
- WebSocket to data-stream.binance.vision (port 443)

Inbound Port (for web access):

- Port 5000 (or custom PORT from .env)

CORS Configuration:

Default: Allows all origins (development mode). For production:

```
app.use(cors({  
  origin: ['https://yourdomain.com'],  
  credentials: true  
}));
```

7. Testing & Validation

7.1 Unit Testing

Backend Functions to Test:

- calcAutoRange — Verify ATR calculation matches expected values
- buildRangeBars — Test bar splitting logic with synthetic klines

- buildVP — Validate POC/VAH/VAL calculation
- detectAbsorptions — Confirm volume/range thresholds
- generateSignal — Check entry/SL/TP levels and R:R ratios

Test Framework: Jest or Vitest

Example Test:

```
import { calcAutoRange } from './range-bar-generator';

test('calcAutoRange returns valid range size', () => {
  const klines = [
    { open: 100, high: 110, low: 95, close: 105, volume: 1000 },
    // ... 13 more klines
  ];
  const range = calcAutoRange(klines);
  expect([5, 10, 25, 50, 100, 200, 500]).toContain(range);
});
```

7.2 Integration Testing

API Endpoint Tests:

- GET /api/config — Returns default config on first run
- POST /api/config — Updates persist to database
- POST /api/analyze — Returns valid analysis object
- GET /api/trades — Returns array (empty or populated)

WebSocket Stream Test:

1. Start live stream: POST /api/live/start
2. Connect to SSE: GET /api/live
3. Verify data messages received within 60 seconds
4. Stop stream: POST /api/live/stop

7.3 Backtesting Validation

Historical Data Test:

1. Download 1 month of 1-minute klines from Binance
2. Run strategy on historical data
3. Record all generated signals

4. Calculate:

- Win rate (% profitable trades)
- Average R:R (mean of winning trades)
- Maximum drawdown
- Sharpe ratio

Expected Performance (BTCUSDT, range=50):

Metric	Target
Win Rate	> 55%
Average R:R	> 1.8
Max Drawdown	< 10%
Sharpe Ratio	> 1.5

Table 3: Expected backtest performance metrics

7.4 UI/UX Testing

Manual Tests:

- Chart renders without errors on Chrome, Firefox, Safari
- Symbol change triggers re-analysis
- Range size selector updates chart
- Live stream toggles correctly (start/stop)
- Settings save persists after page reload
- Trade history displays real/mock trades

Performance Benchmarks:

- Initial chart render: < 500ms
- Live update latency: < 100ms (from WebSocket to render)
- Memory usage stable over 24h live stream (< 500MB growth)

8. Extending the System

8.1 Adding New Indicators

Example: RSI (Relative Strength Index)

Step 1: Add calculation function to strategy-engine.ts:

```
private calcRSI(bars: RangeBar[], period = 14): number {
  if (bars.length < period + 1) return 50;
  let gains = 0, losses = 0;
  for (let i = bars.length - period; i < bars.length; i++) {
    const change = bars[i].close - bars[i - 1].close;
    if (change > 0) gains += change;
    else losses -= change;
  }
  const avgGain = gains / period;
  const avgLoss = losses / period;
  const rs = avgGain / avgLoss;
  return 100 - (100 / (1 + rs));
}
```

Step 2: Integrate into analysis result:

```
const rsi = this.calcRSI(bars);
return { ...result, rsi };
```

Step 3: Display in frontend (e.g., KPI bar or new panel).

8.2 Implementing Live Trading

Step 1: Create Order Management System (OMS)

```
class LiveOMS {
  async createMarketOrder(symbol: string, side: "BUY" | "SELL",
    quantity: number) {
    // Call Binance authenticated API
    const order = await binanceClient.newOrder({
      symbol, side, type: "MARKET", quantity
    });
    return order;
  }
}
```



```

async createStopLoss(symbol: string, side: "SELL" | "BUY", stopPrice:
number, quantity: number) {
  const order = await binanceClient.newOrder({
    symbol, side, type: "STOP_LOSS_LIMIT",
    stopPrice, price: stopPrice * 0.999, // Small slippage
    quantity
  });
  return order;
}

```

Step 2: Integrate with signal generation:

```

if (signal.type === "LONG") {
  const entry = await oms.createMarketOrder(symbol, "BUY",
    quantity);
  const sl = await oms.createStopLoss(symbol, "SELL", signal.sl,
    quantity);
  const tp = await oms.createTakeProfit(symbol, "SELL", signal.tp,
    quantity);
  await createTrade({ ...entry, stopLoss: sl.orderId, takeProfit:
    tp.orderId });
}

```

Step 3: Add order monitoring loop:

```

setInterval(async () => {
  const openTrades = await getOpenTrades();
  for (const trade of openTrades) {
    const slStatus = await binanceClient.queryOrder(trade.stopLoss);
    const tpStatus = await binanceClient.queryOrder(trade.takeProfit);
    if (slStatus.status === "FILLED") {
      await updateTrade(trade.id, { status: "closed", exitPrice: slStatus.price
    });
    }
    // ... handle TP similarly
  }
}, 5000); // Check every 5 seconds

```

Risk Warning: Live trading requires thorough testing, risk controls, and compliance with local regulations.

8.3 Multi-Timeframe Analysis

Approach: Run multiple strategy engines with different range sizes simultaneously.

Implementation:

```
const ranges = [25, 50, 100, 200];
const analyses = await Promise.all(
  ranges.map(r => analyzeWithRange(symbol, r))
);

// Confluence: Take signal only if 2+ timeframes align
const longSignals = analyses.filter(a => a.signal.type === "LONG");
if (longSignals.length >= 2) {
  return { type: "LONG", confidence: 0.9, ... };
}
```

Frontend Display: Add timeframe selector dropdown or multi-chart grid view.

8.4 Machine Learning Enhancement

Use Case: Predict absorption strength using historical features.

Step 1: Feature Engineering

```
interface FeatureVector {
  volumeRatio: number; // volume / avgVolume
  rangeRatio: number; // (high - low) / rangeSize
  deltaRatio: number; // delta / totalVolume
  distanceFromPOC: number; // |price - POC| / (VAH - VAL)
  vwapPosition: number; // (price - VWAP) / stdDev
}
```

Step 2: Collect Training Data

Run backtest and export all absorptions with outcomes (did they lead to profitable trade?).

Step 3: Train Model (Python)

```
from sklearn.ensemble import RandomForestClassifier
```

```

X = df[['volumeRatio', 'rangeRatio', 'deltaRatio', 'distanceFromPOC',
'vwapPosition']]
y = df['profitable'] # 1 if profitable, 0 if not

model = RandomForestClassifier(n_estimators=100)
model.fit(X, y)
model.save('absorption_classifier.pkl')

```

Step 4: Integrate Predictions

Load model in Python backend, expose /predict endpoint, call from strategy engine to filter weak signals.

9. Performance Optimization

9.1 Backend Optimizations

Database Indexing:

```

CREATE INDEX idx_trades_symbol ON trades(symbol);
CREATE INDEX idx_trades_status ON trades(status);
CREATE INDEX idx_trades_opened_at ON trades(opened_at);

```

Connection Pooling (for high load):

Use `better-sqlite3` in WAL mode (already enabled):

```
sqlite.pragma("journal_mode = WAL");
```

Caching Frequently Accessed Data:

```

let configCache: StrategyConfig | null = null;
let cacheExpiry = 0;

export async function getConfig() {
  if (configCache && Date.now() < cacheExpiry) {
    return configCache;
  }
  configCache = await db.select().from(strategyConfig).limit(1).then(r =>
r[0]);
  cacheExpiry = Date.now() + 60000; // 1 minute cache

```

```
return configCache;  
}
```

9.2 Frontend Optimizations

Canvas Rendering:

- Use `requestAnimationFrame` for smooth updates
- Implement dirty region tracking (redraw only changed bars)
- Offscreen canvas for complex layers

React Optimizations:

```
const TradingChart = memo(({ bars, vp, ... }) => {  
  // Component only re-renders when props change  
}, (prev, next) => {  
  return prev.bars.length === next.bars.length &&  
    prev.vp.poc === next.vp.poc;  
});
```

Lazy Loading Components:

```
const TradeHistory = lazy(() => import('./components/TradeHistory'));  
const StrategySettings = lazy(() =>  
  import('./components/StrategySettings'));
```

9.3 Network Optimizations

Compression:

```
import compression from 'compression';  
app.use(compression()); // gzip responses
```

WebSocket Throttling:

```
let lastBroadcast = 0;  
const MIN_INTERVAL = 1000; // Max 1 update/second  
  
ws.on("message", (raw) => {  
  if (Date.now() - lastBroadcast < MIN_INTERVAL) return;  
  // ... process and broadcast  
  lastBroadcast = Date.now();  
});
```

10. Security Considerations

10.1 API Security

Rate Limiting:

```
import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // limit each IP to 100 requests per window
});

app.use('/api/', limiter);
```

Input Validation:

```
router.post("/analyze", async (req, res) => {
  const { symbol, rangeSize } = req.body;
  if (!/\[1-9\]{6,10}$/.test(symbol)) {
    return res.status(400).json({ error: "Invalid symbol" });
  }
  if (rangeSize < 0 || rangeSize > 1000) {
    return res.status(400).json({ error: "Invalid range size" });
  }
  // ... proceed
});
```

10.2 Database Security

Prepared Statements (Drizzle ORM handles automatically):

All queries use parameterized statements, preventing SQL injection.

File Permissions:

```
chmod 600 dev.db # Read/write for owner only
```

10.3 Production Deployment

HTTPS Enforcement:

Use Nginx reverse proxy with Let's Encrypt SSL:

```
server {  
listen 443 ssl http2;  
server_name yourdomain.com;  
ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;  
ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;  
  
location / {  
proxy_pass http://localhost:5000;  
proxy_http_version 1.1;  
proxy_set_header Upgrade $http_upgrade;  
proxy_set_header Connection 'upgrade';  
}  
}
```

Process Management:

Use PM2 for auto-restart and monitoring:

```
pm2 start npm --name "valentini-scalper" -- start  
pm2 save  
pm2 startup
```

11. Troubleshooting Guide

11.1 Common Issues

Issue: "Cannot connect to Binance API"

Solution:

- Check firewall allows outbound HTTPS (port 443)
- Verify [data-api.binance.vision](#) is accessible
- Try alternative mirror: [api.binance.com](#) (requires API key)

Issue: "Range bars not generating"

Solution:

- Verify symbol exists on Binance (use correct format: BTCUSDT)
- Check klines have sufficient data (≥ 14 bars for ATR)
- Increase range size manually if auto-calculation fails

Issue: "Chart not rendering"

Solution:

- Open browser console, check for Canvas errors
- Verify bars array is not empty
- Check canvas dimensions (width/height > 0)

Issue: "Live stream disconnects frequently"

Solution:

- Implement exponential backoff reconnection
- Check network stability
- Reduce update frequency (throttle broadcasts)

11.2 Performance Issues

Symptom: High CPU usage

Diagnosis:

- Profile backend with Node.js inspector: `node --inspect dist/server/index.js`
- Check for infinite loops in strategy engine
- Reduce number of bars analyzed (limit to 500)

Symptom: Slow chart rendering

Diagnosis:

- Reduce bar width (fewer bars visible)
- Disable complex layers (volume bubbles, absorption circles)
- Use WebGL instead of Canvas for 10k+ bars

Symptom: Database locks

Diagnosis:

- Check for long-running transactions
 - Increase WAL checkpoint interval
 - Split reads/writes across separate connections
-

12. Conclusion & Next Steps

12.1 System Capabilities Summary

The Valentini Scalper provides:

- Professional-grade order flow analysis (volume profile, VWAP, absorptions)
- Institutional methodology (Triple-A state machine)
- Real-time data streaming (Binance WebSocket integration)
- Configurable risk management (max losses, R:R ratios, position sizing)
- Persistent trade tracking (SQLite database)
- Production-ready architecture (TypeScript, React, Express)

12.2 Recommended Enhancements

Short-Term (1-2 weeks):

1. Add more symbols (ETH, SOL, XRP pairs)
2. Implement session management (Asian/European/US hours)
3. Add Discord/Telegram alerts for signals
4. Export trades to CSV for analysis

Medium-Term (1-2 months):

1. Multi-timeframe confluence analysis
2. Machine learning signal filtering
3. Backtesting module with walk-forward optimization
4. Mobile-responsive UI (Progressive Web App)

Long-Term (3-6 months):

1. Live trading integration with Binance Spot/Futures
2. Advanced OMS with partial fills, trailing stops
3. Portfolio management (multiple symbols simultaneously)
4. Cloud deployment (AWS/GCP with autoscaling)

12.3 Learning Resources

Order Flow Trading:

- "Trading in the Zone" by Mark Douglas
- "Markets in Profile" by James Dalton
- Fabio Valentini's YouTube channel (Triple-A methodology)

Technical Implementation:

- Binance API Documentation: <https://binance-docs.github.io/apidocs/spot/en/>
- TypeScript Handbook: <https://www.typescriptlang.org/docs/>
- React Performance Optimization: <https://react.dev/learn/render-and-commit>

Quantitative Trading:

- "Algorithmic Trading" by Ernie Chan
- QuantConnect Learn: <https://www.quantconnect.com/learning/>
- Backtrader Documentation: <https://www.backtrader.com/>

Appendix A: Full API Reference

A.1 Backend Endpoints

Method	Endpoint	Description
GET	/api/config	Retrieve current strategy configuration
POST	/api/config	Update strategy parameters (body: StrategyConfig)
POST	/api/analyze	Run analysis on symbol (body: {symbol, rangeSize})
GET	/api/trades	Get trade history (query: ?symbol=BTCUSDT)
GET	/api/live	SSE stream for real-time updates (event-stream)
POST	/api/live/start	Start live streaming (body: {symbol, rangeSize})
POST	/api/live/stop	Stop live streaming

Table 4: API endpoint reference

A.2 Configuration Schema

```

interface StrategyConfig {
symbol: string; // Trading pair (e.g., "BTCUSDT")
rangeSize: number; // 0 = auto, else fixed range
absVolumeMultiplier: number; // Absorption volume threshold (1.5)
absRangeThreshold: number; // Absorption range threshold (0.5)
maxDailyLosses: number; // Stop trading after N losses (3)
riskPerTrade: number; // Risk amount in USD (100)
tpMultiplier: number; // Take-profit multiple of SL (2.0)
minRR: number; // Minimum risk-reward ratio (1.5)
useTripleA: boolean; // Enable Triple-A signals (true)
useAbsorption: boolean; // Enable absorption detection (true)
useVwap: boolean; // Enable VWAP filter (true)
}

```

A.3 Analysis Response Schema

```
interface AnalysisResult {  
  bars: RangeBar[]; // Generated range bars  
  rangeSize: number; // Actual range size used  
  vp: VolumeProfile; // Volume profile with POC/VAH/VAL  
  vwap: number; // Current VWAP value  
  vwapBands: {  
    upper1: number; lower1: number;  
    upper2: number; lower2: number;  
  };  
  delta: number; // Cumulative buy-sell delta  
  deltaSmoothed: number; // 3-bar moving average delta  
  signal: Signal; // Current trading signal  
  absorptions: Absorption[]; // Detected absorption points  
  tripleAPhase: "waiting" | "absorbing" | "accumulating" | "signal";  
  orbHigh: number; // Opening range high  
  orbLow: number; // Opening range low  
  ticker: { price: number; change: number };  
  autoRange: number; // Calculated auto range (if used)  
  orderBook: { bids: OrderBookLevel[]; asks: OrderBookLevel[] };  
}
```

Appendix B: Deployment Checklist

- ☐ Node.js 20+ installed
- ☐ Git repository cloned
- ☐ npm install completed successfully
- ☐ .env file configured (if using custom settings)
- ☐ Firewall allows outbound HTTPS to Binance
- ☐ Port 5000 accessible (or custom PORT)
- ☐ npm run build executes without errors
- ☐ dev.db created automatically on first run
- ☐ Browser opens <http://localhost:5000> successfully
- ☐ Chart renders with default symbol (BTCUSDT)
- ☐ Analyze button triggers data fetch
- ☐ Live stream connects and updates chart
- ☐ Settings save persists after restart
- ☐ Trade history displays correctly

- ☐ SSL certificate installed (production only)
- ☐ PM2 configured for auto-restart (production only)
- ☐ Backups scheduled for dev.db (production only)

Appendix C: Version History

Version	Date	Changes
1.0.0	2026-03-31	Initial release with full Node.js backend, React frontend, Triple-A strategy
1.1.0	TBD	Add multi-timeframe analysis, session filters
1.2.0	TBD	Integrate Python ML module for signal filtering
2.0.0	TBD	Live trading with Binance Spot API

Table 5: Version history

Document Version: 1.0
Last Updated: March 31, 2026
Author: AI Assistant
License: MIT (modify as needed)

Replication Success Rate: Following this guide should enable any developer with intermediate TypeScript/React knowledge to deploy a fully functional Valentini Scalper system within 2-4 hours.
